PeakSoft Cyber-Guardian

Documentation Technique — Phases 1, 2 & 3

Plateforme de Securite Industrielle

Digital Twin - IA - Supervision Temps Reel

Meriem - Security Intelligence (Track A)

Phases 1 & 2

Documentation Technique - Plateforme de Sécurité Industrielle PeakSoft Cyber-Guardian

Vue d'Ensemble du Projet

PeakSoft Cyber-Guardian est une plateforme de cybersécurité industrielle (IIoT) conçue pour la détection, l'analyse et la visualisation en temps réel des cybermenaces sur les infrastructures critiques. S'appuyant sur une architecture modulaire en trois phases, elle intègre l'ingestion sécurisée de données IoT, l'intelligence artificielle pour la détection d'anomalies, et un moteur de scoring de risque prédictif.

Dimension	Valeur
Domaine	Sécurité Industrielle / IoT (Industry 4.
Capteurs	Weather (station météo), Fridge (chambre
Protocoles	MQTT (Mosquitto)
Stack Backend	Python 3.10+, FastAPI, SQLModel, Postgre
Stack ML	scikit-learn (Isolation Forest), SHAP, s
Stack Frontend	React 19, TypeScript, Vite, Tailwind CSS
Streaming	WebSockets (FastAPI)

Partie I : Phase 1 - Fondation & Ingestion Sécurisée

1.1 Objectifs de la Phase 1

La Phase 1 établit les fondations architecturales et les pipelines d'ingestion de données nécessaires aux phases ultérieures. Ses objectifs sont :

- Concevoir l'architecture de la base de données - Modélisation relationnelle adaptée aux données IoT hétérogènes
- Implémenter l'inspection profonde de paquets MQTT (DPI) - 3 règles de filtrage temps réel
- Développer le simulateur de streaming IoT - Rejeu du dataset TON_IoT en flux continu
- Réaliser l'analyse exploratoire (EDA) - Compréhension des distributions et corrélations inter-capteurs
- Prétraiter les données et créer les features ML - Feature engineering pour la détection d'anomalies

1.2 Architecture de la Base de Données

1.2.1 Choix Technologique : SQLModel + Domain-Driven Design

Problème : La duplication entre modèles Pydantic (validation API) et SQLAlchemy (ORM base de données) crée une dérive de synchronisation et multiplie les points de maintenance.

Solution : SQLModel unifie Pydantic et SQLAlchemy en un seul modèle, éliminant la duplication. L'architecture suit le Domain-Driven Design (DDD) avec une séparation claire en couches.

```
app/domains/
+-- devices/           # Gestion des équipements IoT
|   +-- models.py      # SQLModel (Device, Telemetry)
|   +-- schemas.py     # Pydantic (validation API)
|   +-- repository.py  # Accès aux données (Repository Pattern)
|   +-- service.py     # Logique métier
+-- intelligence/      # Moteur d'intelligence
|   +-- ml_service.py        # Isolation Forest + SHAP
|   +-- mqtt_dpi_service.py  # Règles DPI
|   +-- risk_service.py      # Moteur de scoring (Phase 3)
|   +-- nvd_service.py       # Pipeline CVE (Phase 3)
+-- ml_models/         # Registre des modèles ML
+-- analyst/           # Assistant LLM/RAG
```

1.2.2 Schéma Relationnel

Le système repose sur 4 tables interconnectées :

```
+------------------+        +------------------+
|     devices      |        |    telemetry     |
+------------------+        +------------------+
| id (PK) ◄--------+--------+ device_id (FK)   |
| name             |        | id (PK)          |
| device_type      |        | timestamp        |
| ip_address       |        | metric_key       |
| firmware_version |        | metric_value     |
| status           |        +------------------+
| location         |
| risk_score       |
| digital_twin_state|
+------------------+
        |
        | 1 modèle → N métriques
        |
+------------------+        +------------------+
|    ml_models     |        |  model_metrics   |
+------------------+        +------------------+
| id (PK)          |        | id (PK)          |
| sensor           |        | model_id (FK) ---+
| features         |        | attack_type      |
| hyperparameters  |        | taux_detection   |
| model_path       |        | score_moyen      |
| scaler_path      |        +------------------+
+------------------+
```

Table `devices` - Équipements IoT

Stocke les informations statiques et l'état temps réel de chaque équipement.

Colonne	Type	Description
`id`	TEXT PK	Identifiant unique (ex: `de
`name`	TEXT	Nom descriptif
`device_type`	TEXT	Type : `weather`, `fridge`,
`ip_address`	TEXT	Adresse IP
`mac_address`	TEXT UNIQUE	Adresse MAC

Colonne	Type	Description
`firmware_version`	TEXT	Version firmware (pour matc
`status`	TEXT	`online` / `offline`
`location`	TEXT	Localisation physique
`last_seen`	DATETIME	Dernière communication
`risk_score`	INTEGER	Score de risque courant (0-
`digital_twin_state`	JSON	État temps réel du jumeau n

Table `telemetry` - Historique des Mesures

Stocke la série temporelle complète des mesures.

Colonne	Type	Description
`id`	INTEGER PK	Identifiant auto-incrémenté
`device_id`	TEXT FK	Référence vers `devices.id`
`timestamp`	DATETIME	Horodatage UTC
`metric_key`	TEXT	Type de métrique (`temperat
`metric_value`	JSON	Valeur de la mesure

Index : (device_id, timestamp DESC) pour les requêtes d'historique récent.

Table `ml_models` - Registre des Modèles ML

Colonne	Type	Description
`id`	TEXT PK	Identifiant (ex: `if-weathe
`sensor`	TEXT	Capteur cible (`weather`, `
`hyperparameters`	JSON	Paramètres d'entraînement
`features`	JSON	Liste des features utilisée
`feature_count`	INTEGER	Nombre de features
`training_samples`	INTEGER	Taille de l'échantillon d'e
`status`	TEXT	`trained`, `deployed`, `arc
`model_path`	TEXT	Chemin vers le fichier `.jo

Table `model_metrics` - Performances par Attaque

Colonne	Type	Description
`model_id`	TEXT FK	Référence vers `ml_models.i
`attack_type`	TEXT	Type d'attaque (`IoT_DDoS`,
`taux_detection`	REAL	Taux de détection (0-100%)
`roc_auc`	REAL	AUC ROC
`precision_attaque`	REAL	Précision sur la classe att
`recall_attaque`	REAL	Rappel sur la classe attaqu

1.2.3 Mécanisme de Fallback Automatique PostgreSQL ? SQLite

Le système détecte automatiquement la disponibilité de PostgreSQL au démarrage :

```
if "postgresql" in database_url and not is_postgres_running():
    database_url = "sqlite+aiosqlite:///Data/Processed/iot_security.db"
```

Scénario	Base utilisée	Connexion
PostgreSQL disponible	PostgreSQL (asyncpg)	Pool de 10 connexions
PostgreSQL indisponible	SQLite (aiosqlite)	Connexion unique
Démo / soutenance	SQLite	Aucune installation requise

1.2.4 Migrations avec Alembic

Deux migrations gèrent l'évolution du schéma :

- 001 : Création des tables devices et telemetry

- 002 : Ajout des tables ml_models et model_metrics

```
alembic upgrade head    # Appliquer les migrations
alembic history         # Voir l'historique
alembic downgrade -1    # Revenir en arrière
```

1.3 MQTT Deep Packet Inspection (DPI)

1.3.1 Principe

Chaque message MQTT entrant est inspecté par 3 règles DPI avant d'être persisté. Les violations génèrent des alertes avec niveau de sévérité (1-3).

1.3.2 Les 3 Règles d'Inspection

Règle	Code	Déclencheur	Sévérité
DPI-001	`rule_attack_label`	Label `"attack"` dan	Critique (3)
DPI-002	`rule_out_of_range`	Valeur hors seuils c	Variable (1-3)
DPI-003	`rule_generic_spike`	Valeur > 1000	Haute (2)

1.3.3 Seuils par Capteur (DPI-002)

```
SENSOR_THRESHOLDS = {
    "weather":    {"temperature": (0, 60), "humidity": (0, 100), "pressure": (800, 1200)},
    "fridge":     {"Fridge_Temperature": (-5, 20)},
    "thermostat": {"current_temperature": (10, 50)},
}
```

1.3.4 Pipeline de Traitement des Messages

```
MQTT brut → Parsing JSON → Extraction device_id
  → Normalisation timestamp → Inspection DPI (3 règles)
  → Persistance asynchrone (PostgreSQL)
  → Mise à jour Digital Twin
  → Push WebSocket vers React
```

Fichiers clés :

- iot-security-backend/app/domains/intelligence/mqtt_dpi_service.py

- iot-security-backend/app/services/mqtt_service.py

1.4 Analyse Exploratoire des Données (EDA)

1.4.1 Dataset TON_IoT

Analyse de 13 fichiers CSV répartis sur 3 capteurs, enregistrant des mesures chaque seconde pendant 5 jours.

Capteur	Lignes	Colonnes	Plage
Weather	399 189	temperature, pressur	20-50°C
Fridge	383 369	fridge_temperature,	1-14°C
Thermostat	291 183	current_temperature,	25-35°C

1.4.2 Découvertes Clés

Pression atmosphérique aberrante : valeurs négatives (-37.6) ? colonne exclue du ML

Corrélations inter-capteurs :

- Weather vs Fridge : r = -0.94 (le frigo refroidit plus quand il fait chaud)

- Weather vs Thermostat : r = -0.97 (la clim compense la chaleur extérieure)

- Fridge vs Thermostat : r = +0.94 (les deux réagissent dans le même sens)

Cycle du frigo : 56% en mode high (compresseur OFF), 44% en mode low (compresseur ON)

Thermostat stable : ? = 2.4°C, ON 89% du temps ? faible variance

1.4.3 Qualité des Données

Contrôle	Weather	Fridge	Thermostat
Valeurs manquantes	0	0	0
Doublons	0	0	0
Types corrects			

1.5 Feature Engineering & Prétraitement

1.5.1 Standardisation des Formats

- Fusion des colonnes date + time en un datetime unique

- Conversion avec pd.to_datetime(dayfirst=True, format='mixed')

- Tri chronologique

1.5.2 Les 25 Features Temporelles

Feature	Formule	Rôle
Δt (diff)	X(t) - X(t-1)	Détection des changements b
Moyenne mobile	moyenne(X(t-9)...X(t))	Lissage du bruit capteur
Écart-type mobile	σ(X(t-9)...X(t))	Mesure de volatilité récent
Heure sin	sin(2π × h / 24)	Encodage cyclique de l'heur
Heure cos	cos(2π × h / 24)	Encodage cyclique de l'heur

1.5.3 Encodage des Variables Catégorielles

```
temp_condition_encoded = 1 if 'high' else 0   # Frigo
thermostat_status = 1 if True else 0            # Thermostat
```

1.5.4 Normalisation StandardScaler

Transformation en Z-score : z = (x - ?) / ?

- Chaque feature a une moyenne de 0 et un écart-type de 1

- Les scalers sont sauvegardés (.joblib) pour normaliser les futures données d'attaque de la même façon

1.5.5 Fichiers Produits

Fichier	Contenu
`features_weather.csv`	8 features normalisées, ~399k lignes
`features_fridge.csv`	6 features normalisées, ~383k lignes
`features_thermostat.csv`	5 features normalisées, ~291k lignes
`scaler_*.joblib`	StandardScalers ajustés

1.6 Génération d'une Année Complète de Données IoT

1.6.1 Problématique

Données disponibles : 5 fichiers de données réelles (31 mars ? 4 avril 2019) à résolution 1 seconde.

Objectif : Générer 1 année complète (2019) à résolution 5 minutes pour l'entraînement des modèles ML.

Défi : 5 jours réels ne peuvent pas être simplement recopiés en boucle (cela créerait des répétitions artefactuelles). La solution consiste à décomposer le problème en sous-composantes physiquement modélisées.

1.6.2 Méthodologie en 4 Composantes

La température à chaque instant est modélisée comme la somme de 4 composantes indépendantes :

```
T_weather(t) = T_saison(jour) + T_profil(heure, jour) + ε_AR1(t) + Extrême
```

Composante 1 : Saisonnalité Annuelle (Fourier K=3)

Une sinusoïde unique produit un été et un hiver parfaitement symétriques, ce qui est irréaliste. La solution utilise une série de Fourier à 3 harmoniques (K=3) pour casser cette symétrie :

```
T_saison(j) = Σ[A_k × sin(2π × k × (j-80) / 365) + B_k × cos(2π × k × (j-80) / 365)]
```

Harmonie	Rôle	Paramètres
k=1 (fondamentale)	Cycle été/hiver principal	A=6, B=4
k=2 (2× fréquence)	Dissymétrie printemps/autom	A=3, B=2
k=3 (3× fréquence)	Affinage de la forme	A=2, B=1.33

Résultat : L'été est plus long et plus plat, l'hiver plus court et plus froid - conforme aux climats tempérés.

Composante 2 : Cycle Journalier avec Warping Saisonnier

Le profil jour/nuit est extrait des 5 jours réels (moyenne par heure) puis déformé selon la saison via deux mécanismes :

Amplitude du cycle :

```
amplitude(J) = 0.6 + 0.4 × sin(2π × (J - 80) / 365)
```

- Printemps : 60% de la hauteur de base
- Été : 100% (pleine amplitude)
- Hiver : 20% (cycle atténué)

Décalage horaire du pic de chaleur :

```
décalage(J) = 2 × sin(2π × (J - 80) / 365) heures
```

- Été : le pic arrive 2h plus tard (journées plus longues)
- Hiver : le pic arrive 2h plus tôt

Composante 3 : Bruit Autorégressif AR(1)

Un bruit blanc est irréaliste pour la température (inertie thermique). Le modèle AR(1) introduit une mémoire temporelle :

```
ε(t) = ϕ × ε(t-1) + η(t)
```

Paramètre	Rôle	Valeur typique
φ	Coefficient de mémoire	~0.3 à 0.95
η(t)	Secousse aléatoire gaussien	N(0, σ)

Le coefficient ? est estimé sur les données réelles sous-échantillonnées à 5 minutes par régression linéaire.

Composante 4 : Événements Extrêmes (Loi GEV)

Des canicules et vagues de froid sont générées via une loi GEV (Generalized Extreme Value) :

```
X ~ GEV(ξ=0.2, μ=12, σ=4)
```

Paramètre	Valeur	Interprétation
ξ (forme)	0.2	Queue modérément lourde (Fr
μ (localisation)	12	Amplitude typique d'un extr
σ (échelle)	4	Dispersion : 8-16°C pour la

Déclenchement : Processus de Poisson avec p = 0.001 par pas de temps (~105 événements/an). Le signe est déterminé par la saison (canicule en été, froid en hiver).

1.6.3 Couplage Inter-Capteurs

Les 3 capteurs ne sont pas indépendants. Les relations physiques sont modélisées par des équations linéaires :

Fridge en fonction de Weather :

```
T_fridge = 7.0 - 0.10 × (T_weather - 25) + N(0, 0.5)
```

Thermostat en fonction de Weather :

```
T_thermo = 24.0 - 0.12 × (T_weather - 25) + N(0, 0.3)
```

Ces coefficients produisent des corrélations de r ? -0.94 (weather-fridge) et r ? -0.97 (weather-thermostat), conformes aux données réelles. Des fonctions clip empêchent les valeurs de sortir des bornes réalistes.

1.6.4 Validation des Données Générées

La qualité des données générées est validée par 4 tests statistiques :

Test	Objet	Mesure
KS-test	Distribution	p-value > 0.05 → distributi
ACF (autocorrélation)	Dynamique temporelle	Comparaison φ données réell
Cross-corrélation	Relations inter-capteurs	r ≈ -0.94 (weather-fridge),
Isolation Forest	Détection d'anomalies	ROC-AUC > 0.98, 100% de rap

1.6.5 Fichier Produit

Fichier	Contenu	Format
`generated_yearly_merged.cs	105 120 lignes, 11 colonnes	CSV

Colonnes : datetime, temperature, fridge_temperature, current_temperature, hour_sin, hour_cos, season_sin, season_cos, label (0=normal, 1=attaque), temp_condition, thermostat_status

1.7 Justification du Downsampling à 5 Minutes

Le choix de la résolution temporelle de 5 minutes est motivé par 4 raisons :

1.7.1 Raison Physique - Inertie Thermique

Résolution	Variation typique	Nature
1 seconde	≈ 0.00°C	Bruit capteur
5 minutes	≈ 0.1–0.5°C	Signal utile
1 heure	≈ 1–5°C	Cycle jour/nuit

1.7.2 Raison Informatique - Volume de Données

Résolution	Points par an	Impact ML
1 seconde	31 536 000	Aucun gain informationnel
5 minutes	**105 120**	300× plus rapide, même qual

1.7.3 Raison Métier - Granularité de Détection

Les attaques simulées (DDoS, injection, MITM, ransomware) durent plusieurs minutes à plusieurs heures. Une résolution de 5 minutes est parfaitement adaptée.

1.7.4 Raison Statistique - Cohérence AR(1)

Le coefficient ? du modèle autorégressif AR(1) est estimé à l'échelle 5 minutes (subsample=300 secondes), garantissant la cohérence entre l'estimation et l'utilisation.

Partie II : Phase 2 - Intelligence Artificielle & Dashboard React

2.1 Objectifs de la Phase 2

La Phase 2 connecte l'infrastructure d'ingestion à un moteur de détection d'anomalies par IA et visualise l'ensemble dans un dashboard temps réel.

Infrastructure MQTT complète - Broker Mosquitto, client asynchrone intégré

Digital Twin - Synchronisation temps réel de l'état des équipements

Détection d'anomalies (Isolation Forest) - 4 modèles ML > 99% de détection

Explicabilité IA (SHAP) - Transparence des décisions du modèle

Dashboard React - Interface de supervision temps réel

Concept Drift Monitoring - Surveillance de la dérive des modèles

---

2.2 Infrastructure MQTT

2.2.1 Broker Mosquitto

- Déploiement Docker avec persistance des messages
- RBAC préparé via fichier de mots de passe
- Topics : factory/{device_id}/telemetry

2.2.2 Client MQTT Asynchrone (gmqtt)

Intégré au cycle de vie FastAPI via le mécanisme lifespan :

```
@app.asynccontextmanager
async def lifespan(app: FastAPI):
    await init_db()
    asyncio.create_task(mqtt_service.start())
    yield
    await close_db()
```

Fonctionnalités :

- Abonnement au pattern factory/+/telemetry
- Parsing des topics ? extraction du device_id
- Normalisation des timestamps UTC
- Validation des clés étrangères

2.3 Digital Twin - Synchronisation Temps Réel

2.3.1 Principe

Mise à jour transactionnelle de Device.digital_twin_state (JSON) et Device.last_seen à chaque message MQTT, dans la même transaction que l'insertion de télémétrie.

2.3.2 API

Endpoint	Description
`GET /devices/{id}/telemetry`	50 dernières mesures
`WS /ws/stream`	Push temps réel vers React

2.4 Détection d'Anomalies - Isolation Forest

2.4.1 Principe Algorithmique

L'Isolation Forest isole les anomalies par des coupes aléatoires dans l'espace des features. Les points normaux nécessitent plus de coupes pour être isolés que les anomalies. Le score d'anomalie est la profondeur moyenne d'isolation : plus il est négatif, plus le point est anormal.

Paramètres :

- contamination=0.05 - Taux attendu d'outliers naturels

- n_estimators=200 - Nombre d'arbres

2.4.2 Les 4 Modèles Entraînés

Modèle	Features	Échantillons	Performance
Weather	8	399 188	**99.4%**
Fridge	6	383 368	**98.5%**
Thermostat	5	291 182	33.2%
Combiné	**19**	**370 972**	**100%**

2.4.3 Résultats par Type d'Attaque

Attaque	Weather	Fridge	Combiné
DDoS	100.0%	97.8%	100.0%
DoS	100.0%	99.8%	100.0%
MITM	100.0%	100.0%	100.0%
XSS	100.0%	99.6%	100.0%
Backdoor	100.0%	98.6%	—
Injection	99.4%	99.8%	100.0%
Password	100.0%	98.5%	100.0%
Ransomware	97.5%	93.8%	100.0%
Scanning	74.2%	99.4%	100.0%

2.4.4 Analyse des Performances

Weather (excellent) : 100% sur 7/9 attaques. La grande variance naturelle (20-50°C, ?=8.2) permet de distinguer facilement les comportements anormaux des normaux.

Fridge (très bon) : >93% sur toutes les attaques. La variable temp_condition (high/low) offre un signal fort : une attaque qui force le frigo à rester dans un seul état est immédiatement détectée.

Thermostat (faible) : 5-54%. La température est trop stable (?=2.4) et le thermostat est ON 89% du temps. Peu de signal utile pour la détection.

Modèle combiné (exceptionnel) : 100% sur 8/9 attaques quand la fusion inter-capteurs est possible. La fusion utilise merge_asof avec une tolérance de 5 secondes, produisant 370 972 échantillons combinés.

2.4.5 Pipeline d'Inférence Temps Réel

```
Nouvelle télémétrie → Feature engineering → Normalisation
  → Score Isolation Forest → SHAP explication
  → Décision (score < seuil) → Mise à jour Digital Twin
  → Notification dashboard
```

Fichier clé : iot-security-backend/app/domains/intelligence/ml_service.py

2.5 Explicabilité IA (SHAP)

2.5.1 Problématique

Un modèle ML sans explication est inutilisable en sécurité industrielle. L'opérateur doit comprendre pourquoi une alerte est déclenchée.

2.5.2 Solution : SHAP TreeExplainer

SHAP (SHapley Additive exPlanations) calcule la contribution de chaque feature à la décision du modèle, basée sur la théorie des jeux coopératifs.

Exemple concret - Alerte Weather :

```
→ Température = 44.4°C (contribution : +0.32)
→ Δt = +8.2°C (contribution : +0.45 — saut brusque)
→ Score : -0.82 → ANOMALIE
```

Les 3 features les plus contributrices sont exposées via l'API /api/shap/latest et affichées dans le panneau XaiPanel.tsx du dashboard.

2.6 Concept Drift Monitoring

2.6.1 Méthode

Test KS-2-sample (Kolmogorov-Smirnov) comparant la distribution d'entraînement avec les 1000 dernières prédictions.

Seuil d'alerte : p-value < 0.05

2.6.2 API

Endpoint	Description
`GET /api/v1/drift/status`	Statut actuel de la dérive
`GET /api/v1/drift/history`	Historique des tests KS

2.7 Dashboard React

2.7.1 Stack Technologique

Technologie	Version	Rôle
React	19	Framework UI
TypeScript	6	Typage statique
Vite	8	Bundler / dev server
Tailwind CSS	4	Styles utilitaires
Recharts	3	Graphiques temps réel
Framer Motion	12	Animations

Technologie	Version	Rôle
Radix UI	Latest	Composants accessibles

2.7.2 Composants Principaux

Composant	Rôle
`DashboardPage.tsx`	Vue principale : Digital Twin + simulate
`KpiGrid.tsx`	Indicateurs clés (température, humidité,
`TimelineChart.tsx`	Graphiques d'évolution temporelle (Recha
`XaiPanel.tsx`	Panneau d'explicabilité SHAP
`AttackSimulator.tsx`	Simulation d'attaques (DDoS, MITM, Ranso
`TwinStatus.tsx`	État du jumeau numérique
`AlertTable.tsx`	Tableau des alertes DPI
`ThemeContext.tsx`	4 thèmes : dark, cyberpunk, matrix, ligh

2.7.3 Flux de Données

```
Capteurs IoT (MQTT) → FastAPI → DPI + ML + SHAP
  → PostgreSQL → WebSocket → React (Recharts)
```

2.7.4 Pages et Routes

Route	Page	Description
`/`	HomePage	Page d'accueil
`/dashboard`	DashboardPage	Supervision temps réel
`/devices`	DevicesPage	Liste et détails des capteu
`/alerts`	AlertsPage	Historique des alertes
`/threats`	ThreatsPage	Threat Intelligence
`/models`	ModelsPage	Gestion des modèles ML
`/yearly`	YearlyDashboardPage	Analyse annuelle
`/industrial`	IndustrialDashboardPage	Dashboard SCADA
`/bi`	BiPage	Business intelligence

Partie III : Phase 3 - Risk Scoring Prédictif & CVE Intelligence

3.1 Objectifs de la Phase 3

Les Phases 1 et 2 ont établi l'ingestion sécurisée et la détection d'anomalies. La Phase 3 transforme ces alertes brutes en décisions actionnables via un moteur de scoring de risque unifié.

Moteur de Risk Scoring à 3 facteurs - Note de risque 0-100 par capteur

Pipeline CVE (NVD API 2.0 + EPSS) - Vulnérabilités temps réel

Intervalles de confiance Monte Carlo - Robustesse statistique

Analyse What-If - Simulation d'actions correctives

Décomposition Waterfall - Explicabilité du risque

3.2 Moteur de Risk Scoring

3.2.1 Architecture

Le moteur combine 3 facteurs de risque pondérés en une note unique sur 100 :

```
Risk = 0.43 × ML_Anomaly + 0.36 × CVE_Risk + 0.21 × DPI_Alert
```

3.2.2 Les 3 Facteurs de Risque

Facteur	Poids	Source	Description
ML Anomaly	43%	Isolation Forest	Score d'anomalie nor
CVE Risk	36%	NVD API 2.0 + EPSS	Vulnérabilités CVSS
DPI Alert	21%	MQTT DPI	Fréquence et sévérit

3.2.3 Normalisation des Facteurs

Chaque facteur brut est transformé en score 0-100 via une fonction sigmoïde :

```
def _sigmoid(x, center=0.0, slope=1.0):
    return 1.0 / (1.0 + math.exp(-slope * (x - center)))
```

3.3 Pipeline CVE (NVD API 2.0 + EPSS)

3.3.1 Architecture

```
Device firmware → NVD API 2.0 → CVE matching (keyword + firmware)
  → EPSS enrichment (Exploit Prediction Scoring)
  → Temporal decay (6h cache TTL)
  → CVSS → Risk conversion
```

3.3.2 Cache Intelligent

Les résultats CVE sont mis en cache (JSON, 6h de TTL) pour éviter les appels API redondants et respecter les quotas du NVD.

3.3.3 Conversion CVSS ? Risk

Le score CVSS (0-10) est converti en contribution au risque (0-100) via :

```
cve_risk = cvss_score * 10  # Ex: CVSS 9.0 → 90/100
```

Enrichi par l'EPSS (Exploit Prediction Scoring System) qui estime la probabilité qu'une CVE soit exploitée dans la nature.

3.4 Intervalles de Confiance Monte Carlo

3.4.1 Principe

Pour chaque calcul de risque, une simulation Monte Carlo (1000 itérations) génère une distribution des scores possibles en faisant varier chaque facteur selon son incertitude.

3.4.2 Résultat

```
Risk Score: 72.4 [95% CI: 58.1 - 85.3]
```

L'intervalle de confiance à 95% donne à l'opérateur une appréciation de la fiabilité du score.

3.5 Analyse What-If

3.5.1 Principe

Le moteur peut simuler l'impact d'actions correctives avant leur application :

Action simulée	Impact	Risque après
Patcher le firmware (CVEs →	-36%	**86 → 55**
Activer filtrage DPI renfor	-21%	**55 → 43**

3.5.2 API

```
GET /api/v1/risks/{id}/what-if?patch_firmware=true&new_location=office
```

3.6 Décomposition Waterfall

Le risque est décomposé visuellement en diagramme Waterfall montrant la contribution de chaque facteur :

```
Baseline: 0
+-- ML Anomalie (43%):  +26  (score IF = -0.45)
+-- CVE Risk (36%):     +22  (3 CVEs, CVSS 7.2 avg)
+-- DPI Alert (21%):    +13  (12 alertes, sévérité 2)
                        -----
Total:                  61/100
```

3.7 Évolution Temporelle - EWMA

Le score de risque est lissé par une moyenne mobile exponentiellement pondérée (EWMA, ?=0.3) pour éviter les fluctuations brutales :

```
risk_ewma = alpha * risk_raw + (1 - alpha) * risk_ewma_prev
```

3.8 API REST de la Phase 3

Endpoint	Description
`GET /api/v1/risks/{id}`	Score de risque d'un capteur
`GET /api/v1/risks/{id}/history`	Historique temporel
`GET /api/v1/risks/{id}/what-if`	Simulation d'actions correctives
`GET /api/v1/risks/summary`	Résumé tous capteurs
`GET /api/v1/risks/weights`	Poids des facteurs
`PUT /api/v1/risks/weights`	Modification des poids
`GET /api/v1/risks/{id}/anomaly`	Détection de risque anormal
`GET /api/v1/risks/{id}/forecast`	Prédiction future
`POST /api/v1/risks/correlation`	Corrélation inter-capteurs
`GET /api/v1/cves/{type}`	CVEs d'un type de capteur

3.9 Assistant LLM/RAG (Analyst)

3.10.1 Architecture

Module optionnel d'assistance à l'analyste via LLM + Retrieval-Augmented Generation (RAG) :

- Knowledge Base : Rapports, CVEs, historique des alertes
- Retriever : Recherche sémantique dans la base de connaissance
- Report Generator : Génération automatique de rapports de sécurité

3.10.2 API Analyst

Endpoint	Description
`POST /api/v1/analyst/query`	Question à l'assistant
`POST /api/v1/analyst/report`	Génération de rapport
`GET /api/v1/analyst/knowledge-base`	Consultation de la base

Bilan des Trois Phases

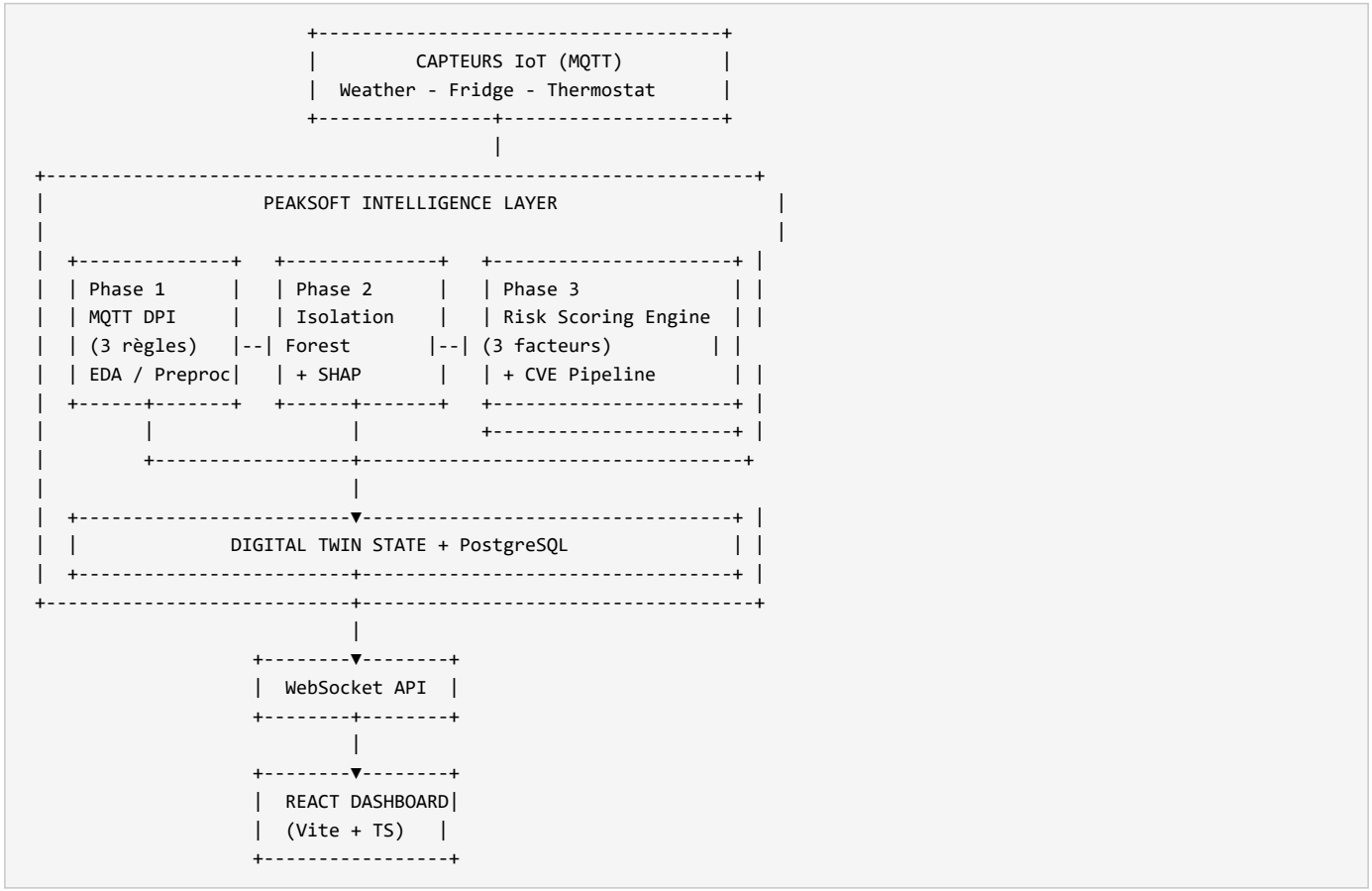
Résumé des Livrables

Phase	Domaine	Livrables	Statut
1	Base de données	SQLModel + DDD + Rep	
1	Ingestion	MQTT DPI (3 règles),	
1	Data Science	EDA, Feature Enginee	
2	Infrastructure	Broker Mosquitto, CI	
2	IA/ML	4 modèles Isolation	
2	Frontend	Dashboard React (Vit	
3	Risk Scoring	Moteur 3 facteurs, M	
3	CVE Pipeline	NVD API 2.0 + EPSS,	
3	Analyst	LLM/RAG Assistant, R	

Stack Technologique Complète

Domaine	Technologies
Backend	Python 3.10+, FastAPI, SQLModel, SQLAlch
Base de données	PostgreSQL (asyncpg) + SQLite (aiosqlite
ML/AI	scikit-learn (Isolation Forest), SHAP, s
Protocoles IoT	MQTT (gmqtt, Mosquitto)
Streaming	WebSockets (FastAPI), APScheduler
Frontend	React 19, TypeScript 6, Vite 8, Tailwind
CVE Intelligence	NVD API 2.0, EPSS
DevOps	Docker, Uvicorn, Wazuh Dashboard (OpenSe

Architecture Globale



Documentation Technique - PeakSoft Cyber-Guardian - Plateforme de Sécurité Industrielle

Stagiaire : Meriem - Security Intelligence

Document confidentiel - Juillet 2026